

Run a change · Linear → implementation → ship

The command cheatsheet. Every code-touching task follows these steps, in order. Steps 0–3 take you from a Linear ticket to written code.

0	Pick the Linear issue <code>roadmap / priority</code> Note its ID — e.g. AGE-123 . Linear says what's next.
1	Propose <code>openspec new change <slug> (or /opsx:propose "idea")</code> Writes <code>proposal.md</code> (+ <code>Linear: AGE-123</code>) · <code>design.md</code> · <code>spec delta</code> · <code>tasks.md</code> .
2	Validate <code>openspec validate --all</code> Green = every requirement has a scenario. A lint, not a review.
2b	Plan-review <code>run ≥2 other-vendor agents on the change</code> They critique <i>proposal</i> + <i>spec delta</i> → <code>changes/<slug>/REVIEWS.md</code> .
GATE	 No code edits until... <code>validate GREEN AND REVIEWS.md ≥ 2</code> Enforced by hook + CI. Emergency escape: <code>GSD_SKIP_REVIEWS=1</code> .
3	Implement <code>/opsx:apply + writing-plans → TDD</code> Failing test first (RED) → make it pass (GREEN). Conditional gates: <code>security</code> · <code>db-sentinel</code> (SQL/RLS) · <code>design+qa</code> (UI).
4	Code-review + verify <code>independent reviewer of the DIFF</code> Then <code>verification-before-completion</code> : the diff satisfies tasks + spec delta.
5	Archive <code>openspec archive <slug> -y</code> Folds the delta into <code>specs/</code> — now current truth; change → <code>changes/archive/</code> .
6	Ship <code>commit · PR · changelog · version · close Linear</code> archive ≠ ship . OpenSpec doesn't commit/PR/tag.

specs/ = current truth · **changes/archive/** = history · **docs/decisions/** = why (ADRs) · **Linear** = what's next · the CLI runs outside any agent → works with `claude` · `codex` · `opencode` · `pi`